

کش لقمه

- سطح: ساده
- طراح: دانیال یاقوتی

در این سوال قصد داریم با توجه مفهوم اینترفیس (Interface) کلاس های زیر را پیاده سازی کنیم:

ساختار فایل ها

```
fileName.zip
├─ Food.java
├─ Pizza.java
├─ Burger.java
├─ Pasta.java
└─ Order.java
```

برای دانلود فایل اولیه پروژه کلیک کنید 

جزئیات پروژه

کلاس Food (از نوع اینترفیس):

این کلاس یک Interface بوده که دارای سه متد زیر می باشد:

۱. متد **String name()** : نام غذا را با فرمت **Food: [Food Name]** برمی گرداند.

۲. متد **double getPrice()** : قیمت برمی گرداند.

۳. متد **void cook()** : باتوجه به نوع غذا یک پیام را چاپ می کند.

کلاس های Pizza , Burger , Pasta :

این سه کلاس سه نوع غذا هستند که Food را پیاده سازی (Implement) می کنند؛ با توجه به مشخصات زیر خروجی مورد انتظار هر متد را مشخص کنید:

کلاس  **Pizza :**

name	price	cook
Pizza	\$120	"Cooking pizza..."

کلاس  Burger :

name	price	cook
Burger	\$95	"Grilling burger..."

کلاس  Pasta :

name	price	cook
Pasta	\$77.5	"Cooking pasta..."

کلاس Order :

این کلاس مسئولیت ثبت یک سفارش را دارد.

سازنده:

```
1 | public Order(food-1, food-2, ..., food-n)
```

یک سفارش شامل n غذا از نوع Food دریافت می‌کند. توجه کنید که استفاده از ArrayList مجاز نیست.

▼ نکته آموزشی

Varargs (مخفف Variable Arguments) به ما اجازه می‌دهد متدی بنویسیم که تعداد متغیری از پارامترها را بپذیرد.

به جای اینکه چندین overload برای تعداد پارامترهای مختلف بنویسیم، می‌توانیم از ... استفاده کنیم.

سینتکس:

```
1 | public void methodName(Type... paramName) {  
2 | }
```

نحوه استاندارد دسترسی به ورودی:

روش دسترسی استاندارد به مقادیر درون Varargs با استفاده از یک حلقه forEach می‌باشد. مثال:

```
1 | public void printNumbers(int... numbers) {  
2 |     for (int n : numbers) {  
3 |         System.out.println(n);  
4 |     }  
5 | }
```

مقدار Varargs می‌تواند همراه پارامترهای معمولی بیاید، ولی باید آخرین پارامتر باشد.

✓ مثال درست:

```
1 | public void printStudent(String name, int... grades)
```

✗ مثال غلط:

```
1 | public void wrongMethod(int... numbers, String name)
```

متدها:

1. متد calculateTotalPrice :

```
1 | public double calculateTotalPrice()
```

هزینه کل فاکتور را محاسبه و برمی‌گرداند.

2. متد pay :

```
1 | public void pay(double money)
```

در صورتی که مبلغ دریافتی مساوی یا بیشتر از هزینه نهایی فاکتور باشد پیغام زیر را چاپ می‌کند:

Pay Success

و در غیر این صورت:

Payment Failed

3. متد isPaid :

```
1 | public boolean isPaid()
```

در صورتی که فاکتور قبلاً پرداخت شده باشد true و در غیر این صورت false برمی‌گرداند.

4. متد printReceipt :

```
1 | public String printReceipt()
```

فاکتور سفارش را به فرمت زیر خروجی می‌دهد:

```
-----Receipt-----  
[Food Name 1]: Quantity $[total Price]  
[Food Name 2]: Quantity $[total Price]  
.  
.  
-----  
Total: $[Total Price]
```

در مورد خروجی این متد به نکات زیر توجه کنید:

- ترتیب آیتم های نمایش داده شده در فاکتور باید به ترتیب Pizza, Burger, Pasta از چپ به راست باشد.
- اگر از یک نوع غذا هیچ سفارشی ثبت نشده بود نباید در فاکتور قرار بگیرد.
- تمام قیمت های چاپ شده تا **دو رقم اعشار** چاپ شوند.

مثال

```
1 public class Main {  
2  
3     public static void main(String[] args) {  
4  
5         Food pizza = new Pizza();  
6         System.out.println(pizza.name());  
7  
8         Food burger = new Burger();  
9  
10        Food pasta1 = new Pasta();  
11  
12        Food pasta2 = new Pasta();  
13  
14        Order order = new Order(pasta1, pizza, burger, new Burger(), pasta2);  
15  
16        System.out.println(order.printReceipt());  
17  
18        order.pay(465);  
19  
20        System.out.println(order.isPaid());  
21  
22        pizza.cook();  
23  
24        burger.cook();  
25  
26    }  
27  
28 }
```

خروجی

```
Food: Pizza  
-----Receipt-----  
Pizza: 1 $120.00  
Burger: 2 $190.00  
Pasta: 2 $155.00  
-----  
Total: $465.00  
Pay Success
```

true

Cooking pizza...

Grilling burger...

جنگ جهانی سوم!

- سطح: متوسط
- طراح: محمد امین مولائی

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

جنگ جهانی سوم آغاز شده است و شما باید فرماندهی جنگ را به عهده بگیرید. هدف اصلی شما حمله به مواضع دشمن است. برای این کار تعدادی ادوات جنگی شامل پهپاد، موشک و جنگنده در اختیار دارید. از آنجا که منابع محدود هستند باید تشخیص دهید برای حمله به هر هدف کدام یک از ادوات جنگی مناسب تر است. همچنین تعداد اهداف منهدم شده دشمن را به دست آورید تا برای برنامه ریزی در حملات بعدی آماده تر شوید!

در صورت لزوم می‌توانید به کلاس ها متد اضافه کنید.

کلاس Target

این کلاس دارای ویژگی های زیر است:

۱. name از نوع String

۲. health از نوع int

۳. isMoving از نوع boolean

هنگامی که health هدف به صفر برسد هدف منهدم میشود.

تنها سازنده (Constructor) این کلاس به شرح زیر است :

```
1 public Target(String name, int health, boolean isMoving) {  
2     //TODO  
3 }
```

همچنین این کلاس دارای یک متد است که هنگام حمله به هدف آسیب را اعمال میکند.

```
1 public void takeDamage(int damage) {  
2     //TODO  
3 }
```

کلاس انتزاعی Unit

تمام واحدهای نظامی از این کلاس ارث‌بری می‌کنند. این کلاس دارای ویژگی‌های زیر است:

۱. name از نوع String

۲. baseDamage یا آسیب پایه از نوع int

۳. ammo (تعداد مهمات) از نوع int

هنگامی که ammo به صفر برسد واحد دیگر قابل استفاده نیست

۴. cost از نوع int و تضمین می‌شود عددی مثبت است

۵. totalDamageDealt از نوع int

مجموع آسیب‌های وارد شده به اهداف از سمت واحد نظامی

۶. attackCount از نوع int

این کلاس سه متد انتزاعی دارد:

```
1 public abstract void attack(Target target);  
2  
3 public abstract int calculateDamage(Target target);  
4  
5 public abstract String getStatusReport();
```

متد calculateDamage آسیب واحد نظامی در هر حمله به هدف را محاسبه می‌کند. متد attack با توجه به میزان آسیب محاسبه شده به هدف آسیب می‌رساند. متد getStatusReport گزارشی از واحد بر می‌گرداند. این متدها باید در کلاس‌های مختلف که در ادامه می‌آیند پیاده‌سازی شوند.

متد امتیازدهی به واحد‌ها:


```
1 public double getScore() {  
2     return (double) totalDamageDealt / cost;  
3 }
```

توجه کنید که این کلاس نیاز به هیچ پیاده سازی یا تغییری **ندارد**.

کلاس Drone

این کلاس از کلاس انتزاعی Unit ارث بری میکند.

۱. متد attack :

اگر پهپاد قابل استفاده بود (ammo باقی مانده بود) حمله میکند و به هدف آسیب میرساند. دقت کنید بعد از هر حمله ammo یک واحد کم میشود. دقت کنید totalDamageDealt بعد از حمله به اندازه آسیب وارد شده به هدف اضافه میشود نه لزوماً آسیبی که واحد میتواند برساند. (برای مثال ممکن است میزان آسیب واحد نظامی از health باقیمانده هدف بیشتر باشد)

۲. متد calculateDamage :

اگر هدف متحرک باشد میزان آسیب در هر حمله برابر میانگین کل آسیب ها (totalDamageDealt) در حملات تاکنون (attackCount) است (اگر تابحال حمله ای صورت نگرفته آسیب را 0 در نظر بگیرید) و در غیر این صورت برابر با آسیب پایه (baseDamage) است.

۳. متد getStatusReport :

اگر مهمات باقی مانده بود عبارت

Drone <drone_name> with base damage <base_damage> is flying in the sky.

و در غیر این صورت عبارت

Drone <drone_name> with base damage <base_damage> is destroyed.

برگردانده میشود.

کلاس Missile

این کلاس از کلاس انتزاعی Unit ارث بری میکند.

۱. متد attack :

اگر موشک استفاده نشده بود حمله میکند و به هدف آسیب میرساند. موشک بعد از یک بار حمله دیگر قابل استفاده نیست!

۲. متد calculateDamage :

میزان آسیب در هر حمله برابر آسیب پایه است.

۳. متد getStatusReport :

اگر مهمات باقی مانده بود عبارت

Missile <missile_name> with base damage <base_damage> is not used.

در غیر این صورت عبارت

Missile <missile_name> with base damage <base_damage> is used.

برگردانده میشود.

کلاس FighterJet

این کلاس از کلاس انتزاعی Unit ارث بری میکند.

۱. متد attack :

اگر جنگنده قابل استفاده باشد حمله میکند و به هدف آسیب میرساند. بعد از انجام حمله میزان baseDamage به میزان 10 واحد افزایش می یابد و ammo یک واحد کم میشود.

۲. متد calculateDamage :

میزان آسیب در هر حمله برابر جمع آسیب پایه و کل آسیب ها (totalDamageDealt) است.

۳. متد getStatusReport :

اگر مهمات باقی مانده بود عبارت

Fighter jet <jet_name> with total damage <total_damage> is cruising in the sky.

در غیر این صورت عبارت

Fighter jet <jet_name> with total damage <total_damage> is destroyed.

برگردانده میشود.

کلاس CommandCenter

این کلاس مرکز فرماندهی حملات و آمارگیری است و دارای ویژگی های زیر است:

۱. آرایه units با حداکثر اندازه 100

۲. آرایه targets با حداکثر اندازه 100

این کلاس دارای متد های زیر است.

```
1 public void addUnit(Unit unit) { //TODO }
2
3 public void addTarget(Target target) { //TODO }
```

با کمک این متد ها واحدها و اهداف به مرکز فرماندهی افزوده میشوند.

```
1 public String executeAttack(Unit unit, Target target) {
2     //TODO
3 }
```

اگر هم واحد نظامی و هم هدف در آرایه های مرکز فرماندهی وجود داشته باشند:

متد attack واحد نظامی فراخوانی میشود.

اگر هدف منهدم شد عبارت

<target_name> under attack. Damage dealt: <damage>. Target destroyed.

و اگر منهدم نشد عبارت

```
<target_name> under attack. Damage dealt: <damage>. Target not destroyed.
```

برگردانده میشود.

در غیر این صورت عبارت

You should add unit/target to command center first.

برگردانده میشود.

توجه کنید که تمام حملات با استفاده از این متد در مرکز فرماندهی انجام میشوند.

```
1 | public Unit recommendUnit(Target target) {
2 |     //TODO
3 | }
```

این متد برای پیشنهاد بهترین واحد نظامی مناسب برای انهدام یک هدف است.

اگر هدف در آرایه مرکز فرماندهی وجود نداشته باشد **null** برگردانده میشود.

در غیر این صورت:

واحدی انتخاب میشود که بتواند در یک حمله، هدف را منهدم کند و همچنین بیشترین امتیاز را داشته باشد. اگر چند واحد این شرایط را داشتند اولین واحد انتخاب میشود.

اگر هیچ واحدی یافت نشد **null** برگردانده میشود.

```
1 | public int getDestroyedTargetsCount() {
2 |     //TODO
3 | }
```

این متد تعداد اهداف منهدم شده را برمیگرداند.

مثال

پروژه را به گونه ای پیاده سازی کنید که با اجرای Main زیر:

```
1 public class Main {
2     public static void main(String[] args) {
3         CommandCenter cc = new CommandCenter();
4         Unit drone = new Drone("Shahed 136", 30, 5, 100);
5         Unit missile = new Missile("Fattah", 50, 1, 200);
6         Unit jet = new FighterJet("Sukhoi su-35", 70, 3, 300);
7         cc.addUnit(drone);
8         cc.addUnit(missile);
9         cc.addUnit(jet);
10
11         Target t1 = new Target("Radar station", 25, false);
12         Target t2 = new Target("Patriot", 60, true);
13         cc.addTarget(t1);
14         cc.addTarget(t2);
15
16         System.out.println(cc.executeAttack(drone, t1));
17         System.out.println(cc.executeAttack(missile, t2));
18         System.out.println(cc.executeAttack(new Drone("Ghost", 20, 3, 50), t1
19
20         System.out.println(drone.getStatusReport());
21         System.out.println(missile.getStatusReport());
22         System.out.println(jet.getStatusReport());
23
24         Unit rec = cc.recommendUnit(t2);
25         System.out.println("Recommended: " + (rec != null ? rec.getName() : "
26     }
27 }
```

خروجی به شکل زیر باشد:

Radar station under attack. Damage dealt: 25. Target destroyed.
Patriot under attack. Damage dealt: 50. Target not destroyed.
You should add unit/target to command center first.
Drone Shahed 136 with base damage 30 is flying in the sky.
Missile Fattah with base damage 50 is used.
Fighter jet Sukhoi su-35 with total damage 0 is cruising in the sky.
Recommended: Shahed 136

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که شامل کلاس های زیر باشد:

Drone.java
Missile.java
FighterJet.java
Target.java
CommandCenter.java

سیستم مدیریت حمل و نقل

- سطح: متوسط
- طراح: رامتین بدری

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

شرکت حمل و نقل نیاز به سیستمی دارد تا انواع وسایل نقلیه خود را مدیریت کند. هر وسیله نقلیه می‌تواند **روشن** یا **خاموش** شود. برخی از وسایل نقلیه نیاز به **سرویس دوره‌ای** دارند. همچنین هر وسیله نقلیه دارای **بازده سوخت** و **وضعیت فنی** متفاوتی است. سرویس‌ها همیشه موفق نیستند و به فیلدهای فنی بستگی دارند. شما باید با استفاده از مفاهیم اینترفیس، کلاس انتزاعی و وراثت، این سیستم را پیاده‌سازی کنید. در صورت لزوم می‌توانید به کلاس‌ها متد اضافه کنید.

اینترفیس Operational

تمام وسایل نقلیه قابلیت روشن و خاموش شدن دارند.

```
1 public interface Operational {  
2     void start();  
3     void stop();  
4     boolean isRunning();  
5 }
```

اینترفیس Maintainable

برخی از وسایل نقلیه باید به صورت دوره‌ای سرویس شوند. سرویس ممکن است موفق یا ناموفق باشد و هزینه دارد.

```
1 public interface Maintainable {  
2     boolean service();  
3     int getServiceDueInDays();  
4     String getServiceReport();  
5     double getServiceCost();  
- }
```

```
}
```

کلاس انتزاعی Vehicle

تمام واحدهای نقلیه از این کلاس ارث‌بری می‌کنند. این کلاس اینترفیس `Operational` را پیاده‌سازی می‌کند و دارای ویژگی‌های زیر است:

۱. `model` : مدل وسیله نقلیه (مانند "Scania K360" یا "Volvo FH16").
۲. `running` : وضعیت روشن بودن وسیله. اگر `true` باشد وسیله روشن است، در غیر این صورت خاموش.
۳. `mileage` : کل کارکرد وسیله بر حسب کیلومتر از ابتدا تا کنون.
۴. `lastServiceMileage` : کارکرد وسیله در زمان آخرین سرویس موفق (واحد: کیلومتر).
۵. `engineHealth` : سلامت موتور به صورت درصدی از 0 تا 100. 100 یعنی کاملاً سالم، 0 یعنی کاملاً از کار افتاده.

سازنده کلاس:

```
1 public Vehicle(String model) {
2     // TODO
3 }
```

متدهای این کلاس باید به صورت زیر پیاده‌سازی شوند:

- `start()` : اگر وسیله خاموش است، آن را روشن کند.
- `stop()` : وسیله را خاموش کند.
- `isRunning()` : وضعیت فعلی را برگرداند.
- `drive(int km)` : افزایش `mileage` به اندازه `km` و کاهش `engineHealth` (هر 1000 کیلومتر 2% کاهش، مقدار سلامت موتور نباید کمتر از صفر شود).

توجه کنید که این کلاس نیاز به تغییر **ندارد** و فقط بخش‌های `// TODO` در آن باید تکمیل شوند.

کلاس Bus

این کلاس از Vehicle ارث‌بری کرده و اینترفیس Maintainable را پیاده‌سازی می‌کند. ویژگی‌های اضافی:

۱. passengerCapacity از نوع int (ظرفیت مسافر)
۲. daysSinceLastService از نوع int (تعداد روز از آخرین سرویس)
۳. SERVICE_INTERVAL_DAYS ثابت برابر ۳۰ روز
۴. fuelEfficiency از نوع double (بازده سوخت (کیلومتر بر لیتر))
۵. tireWear از نوع double (فرسودگی لاستیک (0-100، مقدار اولیه 20.0))
۶. brakePadWear از نوع double (فرسودگی لنت ترمز (0-100، مقدار اولیه 15.0))

سازنده:

```

1 public Bus(String model, int passengerCapacity, double fuelEfficiency) {
2     // TODO
3 }
```

متدها:

- **service()** : موفقیت اگر brakePadWear < 80 و tireWear < 90. در صورت موفقیت: روزها=0، به‌روزرسانی lastServiceMileage، کاهش (30) tireWear، کاهش (40) brakePadWear، افزایش engineHealth (5)
- **getServiceDueInDays()** : اگر $days \geq 30 \rightarrow 0$ ، در غیر این صورت 30-days.
- **getServiceReport()** : فرمت "Bus ... | Capacity: ... | Days since last service: ... | Tire wear: X.X% | Brake wear: Y.Y% | Engine: Z.Z%"
- **getServiceCost()** : نحوه محاسبه هزینه سرویس به این صورت است:

$$200.0 + (tireWear / 10) * 20.0 + (brakePadWear / 10) * 30.0 + ((mileage - lastServiceMileage) / 5000.0) * 50.0$$
- **calculateFuelConsumption(int distance)** : اگر distance کوچکتر مساوی صفر یا fuelEfficiency کوچکتر مساوی صفر بود، صفر برگردانید، در غیر این صورت distance/fuelEfficiency.

متد کمکی (برای شبیه سازی گذر زمان):

```
1 public void addOneDay() {
2     daysSinceLastService++;
3 }
```

کلاس Truck

این کلاس از Vehicle ارث بری کرده و اینترفیس Maintainable را پیاده سازی می کند. ویژگی های اضافی:

۱. loadCapacity از نوع double (ظرفیت بار بر حسب تن)
۲. daysSinceLastService از نوع int (روز از آخرین سرویس (مقدار اولیه 0))
۳. SERVICE_INTERVAL_DAYS ثابت برابر ۴۵ روز
۴. fuelEfficiency از نوع double (بازده سوخت (کیلومتر بر لیتر))
۵. axleHealth از نوع double (سلامت محور (0-100، مقدار اولیه 90.0))
۶. hydraulicFluidLevel از نوع double (سطح روغن هیدرولیک (0-100، مقدار اولیه 80.0))

سازنده:

```
1 public Truck(String model, double loadCapacity, double fuelEfficiency) {
2     // TODO
3 }
```

متدها:

- **service()** : موفقیت اگر $axleHealth \geq 40$ و $hydraulicFluidLevel \geq 30$. در صورت موفقیت: روزها=0، به روزرسانی lastServiceMileage، افزایش (15) axleHealth، افزایش (40) hydraulicFluidLevel، افزایش (8) engineHealth.
- **getServiceDueInDays()** : اگر $days \geq 45 \rightarrow 0$ ، در غیر این صورت 45-days.
- **getServiceReport()** : فرمت "Truck ... | Load capacity: ... tons | Days since last service: ... | Axle health: X.X% | Hydraulic fluid: Y.Y% |"

Engine: Z.Z%"

• **getServiceCost()** : نحوه محاسبه هزینه سرویس به این صورت است:

$$350.0 + ((100.0 - \text{axleHealth}) / 10.0) * 80.0 + ((100.0 - \text{hydraulicFluidLevel}) / 10.0) * 50.0 + ((\text{mileage} - \text{lastServiceMileage}) / 4000.0) * 70.0$$

• **calculateFuelConsumption(int distance)** : مانند Bus.

• متدهای کمکی: `addOneDay`, `getFuelEfficiency`, `damageAxle`, `reduceHydraulicFluid`

کلاس FleetManager

این کلاس مرکز مدیریت ناوگان است و دارای ویژگی‌های زیر می‌باشد:

۱. `vehicles` از نوع `List<Vehicle>` (برای ذخیره وسایل نقلیه)

سازنده:

```
1 public FleetManager() {
2     // TODO
3 }
```

متدها:

```
1 public void addVehicle(Vehicle v) {
2     // TODO
3 }
```

وسیله نقلیه را به لیست اضافه می‌کند.

```
1 public void startAll() {
2     // TODO
3 }
```

همه وسایل نقلیه را روشن می‌کند.

```
1 | public void stopAll() {  
2 |     // TODO  
3 | }
```

همه وسایل نقلیه را خاموش می‌کند.

```
1 | public List<String> getServiceReports() {  
2 |     // TODO  
3 | }
```

از بین وسایل نقلیه، آنهایی که اینترفیس `Maintainable` را پیاده‌سازی کرده‌اند، گزارش سرویس (با `getServiceReport()`) را در یک لیست از رشته‌ها جمع‌آوری کرده و برمی‌گرداند.

```
1 | public int serviceDueCount(int maxDays) {  
2 |     // TODO  
3 | }
```

تعداد وسایل نقلیه `Maintainable` که روزهای باقی‌مانده تا سرویس آنها (براساس `getServiceDueInDays()`) کمتر یا مساوی `maxDays` باشد را برمی‌گرداند.

```
1 | public void performServiceOnAllDue(int maxDays) {  
2 |     // TODO  
3 | }
```

برای تمام وسایل نقلیه `Maintainable` که شرط بالا (`getServiceDueInDays() <= maxDays`) را دارند، متد `service()` را فراخوانی می‌کند.

```
1 | public List<String> getAllBasicInfo() {  
2 |     // TODO  
3 | }
```

برای همه وسایل نقلیه (بدون در نظر گرفتن اینترفیس) متد `getBasicInfo()` را صدا زده و نتیجه را در یک لیست از رشته‌ها برمی‌گرداند.

```
1 | public double getTotalFuelConsumption(int distance) {
```

```

        // TODO
    }

```

جمع مصرف سوخت همه وسایل برای مسافت داده شده.

```

1 | public Vehicle getMostFuelEfficientVehicle() {
2 |     // TODO
3 | }

```

وسیله با بیشترین fuelEfficiency (با استفاده از instanceof و متدهای خاص). اگر لیست خالی است، باید عبارت No Vehicles Found چاپ شود و null برگردانده شود.

```

1 | public void driveAll(int km) {
2 |     // TODO
3 | }

```

برای همه وسایل drive(km) را صدا می‌زند.

مثال

پروژه را به گونه ای پیاده سازی کنید که با اجرای Main زیر:

```

1 | public class Main {
2 |     public static void main(String[] args) {
3 |         FleetManager fm = new FleetManager();
4 |
5 |         Bus bus = new Bus("Scania K360", 50, 5.0);
6 |         Truck truck = new Truck("Volvo FH16", 25.5, 3.0);
7 |
8 |         fm.addVehicle(bus);
9 |         fm.addVehicle(truck);
10 |
11 |         bus.start();
12 |         truck.start();
13 |
14 |         System.out.println("Basic info:");
15 |         for (String info : fm.getAllBasicInfo()) {
16 |

```

```

        System.out.println(info);
    }

    for (int i = 0; i < 19; i++) {
        bus.addOneDay();
        truck.addOneDay();
    }

    System.out.println("\nService reports:");
    for (String report : fm.getServiceReports()) {
        System.out.println(report);
    }

    System.out.println("\nDue in <= 10 days: " + fm.serviceDueCount(10));

    fm.performServiceOnAllDue(10);

    System.out.println("After service:");
    for (String report : fm.getServiceReports()) {
        System.out.println(report);
    }
}
}

```

خروجی به شکل زیر باشد:

Basic info:

Scania K360 - Running | Mileage: 0 km | Engine: 100.0%

Volvo FH16 - Running | Mileage: 0 km | Engine: 100.0%

Service reports:

Bus Scania K360 | Capacity: 50 | Days since last service: 19 | Tire wear: 20.0% | E

Truck Volvo FH16 | Load capacity: 25.5 tons | Days since last service: 19 | Axle h

Due in <= 10 days: 0

After service:

Bus Scania K360 | Capacity: 50 | Days since last service: 19 | Tire wear: 20.0% | E

Truck Volvo FH16 | Load capacity: 25.5 tons | Days since last service: 19 | Axle h

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که شامل کلاس های زیر باشد:

Operational.java
Maintainable.java
Vehicle.java
Bus.java
Truck.java
FleetManager.java

Dispatch

- سطح: متوسط
- طراح: مهدی افشاری

پروژه اولیه را میتوانید از [اینجا](#) دانلود کنید.



سال ۲۰۲۶ است. در شیفت شب مرکز Grid Core، ساعت ۰۲:۱۷ بامداد هشدارها فعال می‌شوند، حمله‌ای هماهنگ و هوشمند که لایه‌های امنیتی شهر را می‌بلعد. رابرت رابرتسون، دیسپچر کهنه‌کاری که همیشه همه چیز را شوخی می‌گرفت، امشب دیگر نمی‌خندد. بودجه فقط برای دو ابزار تخصصی کافی است و پایداری سیستم لحظه‌به‌لحظه فرو می‌ریزد. نبرد تازه شروع شده، اما هنوز فرصت باقی است.

شما به عنوان برنامه‌نویس باید به رابرت رابرتسون کمک کنید و سیستم دیسپچ را برای او طراحی کنید تا بتواند شهر را نجات دهد و با تهدید پیش‌رو (شرود) مقابله کند.





تهدیدها (Incidents)

کلاس IncidentType از نوع Enum

دسته‌بندی تهدیدها را تعریف می‌کند: DDOS RANSOMWARE و ZERO DAY .

کلاس Incident از نوع Interface

هر تهدید (Incident) باید این متودها را پیاده‌سازی کند:

- `String id()` : شناسه یکتای تهدید را برمی‌گرداند.
- `IncidentType type()` : نوع تهدید را برمی‌گرداند.
- `int severity()` : شدت فعلی تهدید (بین ۰ تا ۱۰۰) را برمی‌گرداند.
- `int timeToBreach()` : تعداد گام‌های باقی‌مانده تا نفوذ به سیستم را برمی‌گرداند.
- `boolean resolved()` : اگر تهدید دیگر فعال نباشد مقدار `true` برمی‌گرداند.
- `int x()` : موقعیت `x` تهدید را بر می‌گرداند.
- `int y()` : موقعیت `y` تهدید را بر می‌گرداند.
- `void tick()` : منطق به‌روزرسانی تهدید در هر گام زمانی را اجرا می‌کند.
- `void mitigate(int power)` : شدت تهدید را بر اساس قدرت ابزار دفاعی کاهش می‌دهد.
- `void amplify(int power)` : شدت تهدید را بر اساس قدرت ابزار دفاعی افزایش می‌دهد.
- `void spread(Dispatcher dispatcher)` : منطق انتشار تهدید به همسایگان را پیاده‌سازی

می‌کند.

کلاس انتزاعی BaseIncident

این کلاس رابط Incident را پیاده‌سازی کرده و داده‌های مشترک و منطق اصلی را مدیریت می‌کند. (انواع تهدید ها باید از این کلاس ارث بری کنند.)

• فیلدهای Protected:

- id ◦
- type ◦
- severity ◦
- timeToBreach ◦
- age ◦
- x, y ◦
- resolved ◦
- breached ◦

• **سازنده (Constructor):** مقداردهی اولیه تمامی فیلدها.

• **متود tick():** در هر گام مقدار timeToBreach را یک واحد کاهش و مقدار age را یک واحد افزایش می‌دهد.

• **نکته:** اگر تهدید resolved باشد، این متود هیچ کاری انجام نمی‌دهد.

• **متود mitigate(int power):** مقدار power را از severity کم می‌کند. اگر severity به صفر رسید وضعیت resolved را به true تغییر می‌دهد. **نکته:** مقدار severity نمیتواند منفی شود. **نکته:** شروط مربوط به severity را با متود resolveIfDone پیاده سازی کنید.

• **متود amplify(int power):** مقدار power را به severity اضافه می‌کند. **نکته:** مقدار severity نمیتواند مقداری بزرگتر از 100 بگیرد.

• **متود markBreached():** وقتی timeToBreach به صفر می‌رسد، این متود فراخوانی می‌شود. این متود باید همزمان breached = true و resolved = true قرار دهد؛ زیرا حمله‌ای که انجام شده دیگر «فعال» نیست، اما به جای حل شدن، با شکست مواجه شده است.

• **نکته:** این متود تنها در متود run از کلاس Dispatcher مورد استفاده قرار می‌گیرد!

کلاس: DdosIncident

- **متود tick()** : در هر گام severity آن **۱ واحد** افزایش می‌یابد (حداکثر تا ۱۰۰).
- **نکته:** اگر تهدید resolved باشد، این متود هیچ کاری انجام نمی‌دهد.
- **متود mitigate()** : طبق رفتار پیش فرض عمل میکند.
- **متود spread()** : شدت تمامی تهدیداتی که در همسایگی این تهدید هستند، به اندازه 2 واحد افزایش می‌یابند.
- **نکته:** تشخیص همسایگی در کلاس Dispatcher صورت می‌گیرد!

کلاس: RansomwareIncident

- **متود tick()** : در هر گام severity آن **۳ واحد** افزایش می‌یابد (حداکثر تا ۱۰۰).
- **نکته:** اگر تهدید resolved باشد، این متود هیچ کاری انجام نمی‌دهد.
- **متود mitigate()** : به دلیل رمزنگاری فایل‌ها قدرت هر ابزاری روی این تهدید **نصف** می‌شود (مثلاً: $\text{power} / 2$).
- **متود spread()** : در صورت زوج بودن age یک تهدید جدید از نوع RansomwareIncident در سمت راست موقعیت فعلی ایجاد میشود. با شدتی نصف شدت همین تهدید و timeToBreach برابر با 4 ثانیه!
- برای ایجاد از متود spawn در کلاس Dispatcher استفاده کنید. (تنها کاربر spawn همینجاست در جاهای دیگه باید از متود add استفاده شود!)

کلاس: ZeroDayIncident

- **فیلد اضافی:** boolean analyzed (مقدار پیش فرض false).
- **متود tick()** :
- اگر **آنالیز نشده باشد** تغییرات severity بر اساس یک چرخه ۳ مرحله‌ای با فرمول $\text{age} \% 3 - 1$ است:
 - گام 0: $\text{severity} += 4$
 - گام 1: $\text{severity} -= 2$
 - گام 2: $\text{severity} += 1$

- اگر **آنالیز شده باشد** severity در هر گام به صورت ثابت **یک واحد** افزایش می‌یابد.
- **نکته:** اگر تهدید resolved باشد، این متود هیچ کاری انجام نمی‌دهد.
- **متود spread():** اگر آنالیز شده باشد به شدت همه همسایگان خود یک واحد اضافه میکند.
- **متود ویژه analyze():** وضعیت آنالیز را به true تغییر می‌دهد.
- **متود ویژه transformIfAnalyzed(String newId):** اگر تهدید آنالیز شده باشد:
- اگر severity ≥ 50 باشد یک تهدید جدید از نوع RansomwareIncident و timeToBreach برابر با 5 برمی‌گرداند.
- اگر severity < 50 باشد یک تهدید جدید از نوع DdosIncident و timeToBreach برابر با 5 برمی‌گرداند.
- در غیر این صورت null بر می‌گرداند.
- تهدید اصلی (ZeroDay) باید به عنوان resolved علامت‌گذاری شود.

ابزار های دفاعی (Tools)

کلاس ResponseTool از نوع Interface

رفتار ابزارها را تعریف می‌کند:

- name()
- baseCost()
- costPerUnit()
- cooldownValue(): زمان انتظار (به گام) برای آماده‌شدن دوباره ابزار پس از استفاده را برمی‌گرداند.
- ready()
- supports(Incident incident)
- x()
- y()
- useOn(Incident incident)
- tick()

۱. زیر کلاس Tools.Scrubber

• مشخصات فنی:

- مشخصه (name): TrafficScrubber
- هزینه پایه (baseCost): 15 واحد
- هزینه به ازای هر واحد طی شده (costPerUnit): 1 واحد
- زمان انتظار (cooldownValue): 2 گام
- متود ready(): زمانی true بر میگرداند که currentCooldown به صفر برسد.

• منطق پشتیبانی (supports):

- اگر بتواند روی یک حمله اثر گذار باشد مقدار درست بر میگرداند.
- این ابزار به صورت پیش فرض برای حملات DDOS طراحی شده است.
- نکته حیاتی: این ابزار نمیتواند روی یک حمله ZERODAY عمل کند، مگر اینکه آن حمله قبلاً توسط ابزار دیگری (مانند Isolator) آنالیز شده باشد.

• نحوه عملکرد (useOn):

- قدرت کاهش شدت (mitigate) این ابزار برابر با 12 واحد است.
- پس از هر بار استفاده، شمارنده currentCooldown برابر با مقدار cooldownValue (یعنی 2) قرار می گیرد.
- متد tick(): در هر مرحله از شبیه سازی، اگر ابزار در حال استراحت باشد (currentCooldown > 0)، یک واحد از زمان انتظار آن کم می کند.

۲. زیر کلاس Tools.Isolator

این کلاس ابزاری پیشرفته تر و گران تر است که نقش کلیدی در شناسایی تهدیدات ناشناخته دارد.

• مشخصات فنی:

- مشخصه (name): IsolationProtocol
- هزینه پایه (baseCost): 25 واحد
- هزینه به ازای هر واحد طی شده (costPerUnit): 2 واحد
- زمان انتظار (cooldownValue): 3 گام

• منطق پشتیبانی (supports):

- اگر بتواند روی یک حمله اثر گذار باشد مقدار درست بر میگردداند.
- این ابزار از حملات **RANSOMWARE** و **ZERODAY** پشتیبانی می‌کند.
- برخلاف Scrubber، این ابزار نیازی به آنالیز قبلی ندارد و می‌تواند مستقیماً روی Zero-Day های ناشناخته عمل کند.
- **نحوه عملکرد (useOn):**
 - قدرت کاهش شدت (mitigate) این ابزار برابر با **20 واحد** است.
 - **قابلیت ویژه (Analysis Trigger):** اگر هدف یک **ZERODAY** باشد، این ابزار بلافاصله متد `analyze()` را روی آن فراخوانی می‌کند.
 - پس از استفاده، شمارنده `currentCooldown` به مقدار `cooldownValue` (یعنی 3) بازنشانی می‌شود.
- **متد tick() :** مشابه ابزار قبلی، وظیفه مدیریت زمان استراحت ابزار را بر عهده دارد.

کلاس Dispatcher

کلاس Dispatcher وظیفه ردیابی **System Integrity** (شروع از 1000) و **Budget** (شروع از 150) را بر عهده دارد.

- یک آرایه از تهدیدات در حال وقوع (active) و یک آرایه مربوط به ابزار ها داریم. مدیریت این دو آرایه به عهده شماست.
- تعداد خانه های پیش فرض این آرایه 10 میباشد. و اگر نیاز به تعداد بیشتری بود به صورت داینامیک تعداد آن را افزایش دهید.

سازنده (Constructor):

مقداردهی اولیه تمامی فیلدها.

- مقدار اولیه `nextId` را برابر یک قرار دهید.

متود addIncident(Incident incident) :

- یک تهدید به تهدیدات اضافه میکند.

متود `addTool(ResponseTool tool)` :

- یک ابزار به ابزارها اضافه میکند.

متود `pickBestIncidentIndex()` :

در میان تهدیدهای فعال (حل نشده) می‌گردد و ایندکس «مناسب‌ترین» تهدید برای اعزام ابزار را بر اساس یک سیستم امتیازدهی ترکیبی پیدا می‌کند. الگوریتم امتیازدهی برای هر تهدید:

۱. فقط تهدیدهایی بررسی می‌شوند که:

- نباید resolved شده باشد.
- حداقل یک ابزار آماده و مناسب (با بودجه کافی) برای آن‌ها وجود داشته باشد.

۲. محاسبه امتیاز:

- $urgency = 1.0 / (timeToBreach + 1)$
- $severity$ = فعلی تهدید $severity$
- $spreadRisk = 5 \times$ تعداد همسایه‌های حل نشده
- $distancePenalty = 2 \times$ حداقل فاصله تا نزدیک‌ترین ابزار

• امتیاز نهایی:

- $score = 50 \times urgency + 2 \times severity + spreadRisk - distancePenalty$

۳. تهدیدی با بیشترین امتیاز انتخاب می‌شود.

۴. اگر هیچ تهدید مناسبی یافت نشد، مقدار 1- برگردانده می‌شود.

متود `pickToolIndexFor(Incident inc)` :

در آرایه ابزارها می‌گردد و بهترین ابزار را برای تهدید داده شده پیدا می‌کند. شرایط لازم برای یک ابزار:

۱. آماده باشد

۲. از نوع تهدید پشتیبانی کند

۳. هزینه کل استفاده از بودجه فعلی بیشتر نباشد.

محاسبه هزینه:

- $distance = |inc.x - tool.x| + |inc.y - tool.y|$
- $totalcost = tool.baseCost() + distance \times tool.costPerUnit()$

• امتیازدهی برای انتخاب بهترین ابزار:

- $score = (incidentseverity/totalcost) - currentCooldown$

• ابزاری با بیشترین امتیاز انتخاب می‌شود. اگر هیچ ابزاری یافت نشد، مقدار 1- برگردانده می‌شود.

متود `removeResolved()`:

- بعد از هر گام، باید آرایه حملات را پاک‌سازی کند. حملاتی که یا `resolved` شده‌اند یا `breached` باید از آرایه حملات فعال حذف شوند تا در گام بعدی پردازش نشوند.
- اگر تهدید `resolved` باشد و $severity \leq 0$ (یعنی واقعاً خنثی شده باشد): **پاداش** دریافت می‌کنیم.
- اگر تهدید `resolved` باشد ولی $severity > 0$ (یعنی نفوذ کرده): پاداشی دریافت نمی‌کنیم.
- **نکته:** برای دریافت پاداش از متود `reward` استفاده شود.

متود `newId()`:

- یک متود کمکی خصوصی که با استفاده از یک شمارنده (`Counter`)، شناسه‌هایی مثل `INC-1` , `INC-2` و... تولید می‌کند (مخصوصاً برای زمان‌هایی که یک `ZeroDay` تغییر شکل می‌دهد و نیاز به ID جدید دارد).

متود `run(int steps)`:

در هر گام (`Step`) موتور باید این اقدامات را دقیقاً به همین ترتیب انجام دهد:

۱. به‌روزرسانی تهدیدها: فراخوانی `tick()` برای تمام تهدیدهای فعال.

۲. فراخوانی متود `spread()`:

○ یک snapshot از activeCount فعلی میگیرد (چون ممکن است در حین spread تهدید جدید اضافه شود).

○ برای تهدیدهای 0 تا snapshot-1: متد spread() را فراخوانی میکند.

۳. بررسی نفوذ (Breach):

○ برای تهدیدهای 0 تا snapshot-1 که breached نشده اند و $\text{timeToBreach} \leq 0$:

○ اگر DDOS باشد: $5 \times \text{severity}$

○ اگر RANSOMWARE باشد: $10 \times \text{severity}$

○ اگر ZERODAY باشد: $8 \times \text{severity}$ از کل integrity سیستم کسر می‌شود.

۴. تغییر شکل (Transformation):

○ یک snapshot از activeCount فعلی بگیر.

○ تمام ZeroDayIncident ها را چک کنید. اگر آنالیز شده‌اند آن‌ها را تبدیل کرده و تهدید

جدید را به آرایه فعال اضافه کنید.

۵. به‌روزرسانی ابزارها:

○ فراخوانی متود tick() برای تمامی ابزار ها و به‌روزرسانی currentCooldown در صورت

نیاز

۶. عملیات اعزام ابزار:

○ **انتخاب هدف:** خطرناک ترین هدف را انتخاب کنید.

○ **انتخاب ابزار:** بهترین ابزار را انتخاب کنید.

○ **اجرا:** ابزار را اعمال کنید هزینه را کسر کرده و زمان انتظار (currentCooldown) را تنظیم کنید.

۷. **پاک‌سازی:** متود متناظر با پاک سازی را فراخوانی کنید.

۸. در نهایت در آخر حلقه:

اگر integrity به صفر برسد و یا تمامی تهدید ها خنثی شوند حلقه به پایان میرسد. و پیغامی

مطابق با شکست یا پیروزی چاپ میشود:

○ پیغام پیروزی:

"Status: SUCCESS | Final Integrity: " + integrity + " | Remaining Budge"

در غیر این صورت:

○ پیغام شکست چاپ میشود:

"Status: FAILED | Final Integrity: " + integrity + " | Remaining Budget

متود `getNeighbors(int x, int y)`:

- تمام تهدید هایی که فعال و حل نشده هستند و فاصله منهتنی برابر با 1 داشته باشند را بر میگرداند.

نکته: فاصله منهتنی: $|x1 - x2| + |y1 - y2|$

متود `boolean hasIncidentAt(int x, int y)`:

- بررسی میکند در موقعیت داده شده تهدیدی وجود دارد یا خیر

متود `void spawn(Incident inc)`:

- تنها در صورتی که به سقف تهدید های فعال حال حاضر نرسیده باشیم، تهدید جدیدی اضافه میکند. (آرایه resize نمیشود!)

متود `int distance(Incident inc, ResponseTool tool)`:

- فاصله منهتنی بین تهدید و ابزار را محاسبه می‌کند:

$$distance = |inc.x - tool.x| + |inc.y - tool.y|$$

متود `int reward(Incident inc)`:

- اگر تهدید از نوع Ddos باشد 5 واحد را بر میگرداند.
- اگر تهدید از نوع Ransomware باشد 10 واحد را بر میگرداند.
- اگر تهدید از نوع ZeroDay باشد 20 واحد را بر میگرداند.

متود `int minDistanceToAnyTool(Incident inc)`:

- حداقل فاصله منهتنی تهدید تا نزدیکترین ابزار (از هر نوعی) را برمی‌گرداند.
- اگر هیچ ابزاری وجود نداشته باشد، 0 برمی‌گرداند.

ساختار فایل

فایل ارسالی باید یک فایل .zip شامل کلاس‌های زیر باشد:

- Incident.java
- IncidentType.java
- BaseIncident.java
- DdosIncident.java
- RansomwareIncident.java
- ZeroDayIncident.java
- ResponseTool.java
- Tools.java
- Dispatcher.java

درخواست مرخصی

- سطح: سخت

- طراح: سیدمحمد حسینی

علیرضا که به تازگی در یک شرکت نرم‌افزاری مشغول به کار شده، در همان روزهای اول متوجه شد که حوصله‌ی زندگی کارمندی را ندارد، برای همین تصمیم گرفت تا از رئیس مرخصی بگیرد! اما رئیس به این راحتی‌ها با او راه نمی‌آید و شرط گذاشته که تنها در صورتی با مرخصی امیر موافقت می‌شود که او یک **سرویس مرخصی** برای کارمندهای شرکت پیاده سازی کند. حالا علیرضا از شما کمک می‌خواهد تا زودتر به روزهای خوش‌گذرانی‌اش برسد.

جزئیات برنامه

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

روال کار به این صورت است که هر کارمند درخواست مرخصی خود را برای یک کارفرما ارسال میکند که ممکن است این درخواست پذیرفته یا رد شود. هر عملیاتی که کارمند یا کارفرما انجام بدهد، یک تسک برای آنها به حساب می‌آید که در هر بخش بصورت دقیق‌تر توضیح داده شده.

کلاس انتزاعی Person

همه‌ی اعضای شرکت از کلاس انتزاعی Person ارث‌بری میکنند، که دارای ویژگی‌های زیر است:

۱. ویژگی name از جنس String

۲. ویژگی id از جنس Long

۳. آرایه‌ای از جنس Task (که در ادامه توضیح می‌دهیم)

۴. شمارنده تسک‌ها از جنس int

- ظرفیت آرایه تسک‌ها برای هر شخص **حداکثر ۱۰۰ تسک** است. شما باید این محدودیت را در کدهای خود مدیریت کنید؛ به این معنی که اگر ظرفیت پر شده بود و تسک جدیدی به شخص ارجاع داده شد، متد doTask نباید کرش کند و باید مقدار false برگرداند.

دقت کنید که id هر شخص منحصر به فرد است و **فقط یکبار تعیین میشود**؛ برای فیلدهای مورد نیاز getter مناسب تعریف کنید.

تنها سازنده (Constructor) این کلاس به شرح زیر است :

```
1 | public Person(Long id, String name) {
2 |     //TODO
3 | }
```

این کلاس یک متد انتزاعی بصورت زیر نیز دارد:

```
1 | public /* TODO */ boolean doTask(Task task);
```

کلاس Task

که دارای فیلدهای زیر است:

۱. فیلد title از جنس String
 ۲. فیلد isDone از جنس boolean که بیانگر انجام شدن تسک است.
 ۳. فیلد taskOwner از جنس Person
 ۴. فیلد leaveReq از جنس LeaveReq (که در ادامه توضیح می‌دهیم)
- سازنده (Constructor) این کلاس به صورت زیر است :

```
1 | public Task(String title, Person taskOwner) {
2 |     //TODO
3 | }
```

توجه: برای تمام فیلدهای پرایویت این کلاس getter, setter مناسب پیاده‌سازی کنید.

کلاس LeaveReq

که دارای فیلدهای زیر است:

۱. فیلد employee از جنس Person

۲. فیلد employer از جنس Person

۳. فیلد isAccepted از جنس boolean که بیانگر پذیرفته شدن درخواست مرخصی است.

۴. فیلد استاتیک allRequests، آرایه‌ای از جنس LeaveReq است که تمام درخواست‌های مرخصی ساخته شده را شامل می‌شود.

۵. فیلد leaveReqCount از جنس int که شمارنده درخواست‌های موجود در آرایه allRequests می‌باشد.

• ظرفیت کل درخواست‌های سیستم حداکثر ۱۰۰۰ عدد است.

• **توجه بسیار مهم:** در این تمرین شما به هیچ‌وجه مجاز به استفاده از Collection‌ها نیستید و

پیاده‌سازی‌ها باید صرفاً با استفاده از آرایه‌های استاندارد جاوا انجام شود.

سازنده (Constructor) این کلاس به صورت زیر است :

```
1 public LeaveReq(Person employee, Person employer){
2     //TODO
3 }
```

توجه: برای تمام فیلدهای پرایویت این کلاس getter, setter مناسب پیاده‌سازی کنید.

این کلاس دارای یک متد toString به شکل زیر است که باید اطلاعات هر درخواست را برگرداند(به نمونه‌های موجود در مثال توجه کنید)

```
1 //TODO
2 public String toString(){
3     //TODO
4 }
```

کلاس Employer

این کلاس از کلاس انتزاعی Person ارث‌بری میکند و فقط همان سازنده را صدا می‌زند.

```
1 public Person(Long id, String name) {
2     //TODO
3 }

1 public void acceptLeaveReq(LeaveReq req) {
2 }
```

```

        //TODO
    }
    public void declineLeaveReq(LeaveReq req) {
        //TODO
    }

```

که باید درخواست داده شده را به ترتیب تایید یا رد کند.

```

1 | //TODO
2 | public boolean doTask(Task task) {
3 |     //TODO
4 | }

```

پیاده‌سازی این متد انتزاعی به این صورت است که در مرحله اول باید تشخیص داده شود که تسک ورودی برای این کارفرما (با همان id) تعریف شده است یا خیر. اگر تسک مربوط به کارفرما نباشد، متد `false` برمی‌گرداند.

در غیر این صورت، یعنی تسک متعلق به این کارفرما است. از آنجایی که کارفرما علاوه بر کارهای عادی، مسئول بررسی مرخصی‌ها نیز هست، سیستم باید بررسی کند که آیا این تسک حامل یک درخواست مرخصی است یا خیر (یعنی باید بررسی کنید فیلد `leaveReq` داخل تسک مقدار دارد یا `null` است).

• اگر تسک دارای درخواست مرخصی بود: کارفرما باید سابقه کارمندی که درخواست را ارسال کرده

است (یعنی شیء `employee` داخل همان درخواست مرخصی) را بررسی کند! اگر آن **کارمند** تا به حال حداقل یک تسک انجام داده باشد، با درخواست مرخصی او موافقت می‌شود (صدا زدن متد مربوطه) و در غیر این صورت درخواست مرخصی‌اش رد می‌شود.

• اگر تسک عادی بود (درخواست مرخصی نداشت): نیازی به بررسی مرخصی نیست و کارفرما صرفاً خود تسک را انجام می‌دهد.

در هر دو حالت بالا: این تسک برای کارفرما انجام شده محسوب می‌شود. بنابراین، باید این تسک به لیست تسک‌های کارفرما اضافه شده و در نهایت متد `true` برگرداند.

نکته بسیار مهم: ظرفیت آرایه تسک‌های هر شخص حداکثر ۱۰۰ تسک است. کدهای شما باید در برابر پر شدن ظرفیت آرایه مقاوم باشند؛ به این معنی که پیش از اضافه کردن تسک جدید، ظرفیت بررسی شود. اگر ظرفیت آرایه تسک‌های کارفرما پر شده بود، تسک جدید اضافه نمی‌شود و متد باید `false` برگرداند. برنامه

شما تحت هیچ شرایطی نباید با خطای پر شدن آرایه یا `NullPointerException` متوقف شود.

کلاس Employee

این کلاس از کلاس انتزاعی `Person` ارث‌بری میکند و فقط همان سازنده را صدا میزند.

```
1 public Person(Long id, String name) {  
2     //TODO  
3 }
```

که باید یک رشته شامل همه‌ی درخواست‌های مرخصی که توسط این کارمند ارسال شده را با متد `toString` که در کلاس `LeaveReq` تعریف کردیم، برگرداند. دقت کنید که در هر خط از این رشته باید اطلاعات یک درخواست ذخیره شود.

```
1 //TODO  
2 public boolean doTask(Task task) {  
3     //TODO  
4 }
```

پیاده سازی این متد انتزاعی به این صورت است که در مرحله اول باید تشخیص داده شود که تسک ورودی برای این کارمند(با همان `id`) تعریف شده یا نه، اگر مربوط به کارمند نباشد `False` برمیگرداند، در غیر این‌صورت تسک برای کارمند انجام شده، این تسک به لیست تسک‌های این کارمند اضافه شده و درنهایت متد `True` برمی‌گرداند.

مثال

پروژه را به گونه ای پیاده سازی کنید که با اجرای `Main` زیر:

```
1 public class Main {  
2     public static void main(String[] args) {  
3         Employee employee1 = new Employee(1L, "arash" );  
4         Employer employer1 = new Employer(2L, "Sina" );  
5         LeaveReq req1 = new LeaveReq(employee1, employer1);  
6         Task task1 = new Task("kare avale emp1", employee1);  
7         Task task2 = new Task("morakhasi avale emp1", employer1);  
8         task2.setLeaveReq(req1);  
9     }  
10 }
```



```

        employee1.doTask(task1);
        employer1.doTask(task2);

        Employee employee2 = new Employee(3L, "mahsa" );
        Employer employer2 = new Employer(4L, "mohsen" );
        LeaveReq req2 = new LeaveReq(employee2, employer2);
        Task task3 = new Task("morakhasi avale emp2", employer2);
        task3.setLeaveReq(req2);
        employer2.doTask(task3);
        System.out.println(employee2.printAllRequests());

        LeaveReq req3 = new LeaveReq(employee1, employer2);
        Task task4 = new Task("morakhasi dovome emp1", employer2);
        task4.setLeaveReq(req3);
        employer2.doTask(task4);
        System.out.println(employee1.printAllRequests());
    }
}

```

خروجی به شکل زیر باشد:

```

1 | LeaveReq requested by employee: mahsa with id:3 was declined by employer: moh
2 | LeaveReq requested by employee: arash with id:1 was accepted by employer: Sin
3 | LeaveReq requested by employee: arash with id:1 was accepted by employer: moh

```

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که شامل کلاس های زیر باشد:

Employee.java
 Employer.java
 LeaveReq.java
 Person.java
 Task.java

کارخانه داری (امتیازی)

- سطح: امتیازی
- طراح: حدیث صابری

علی در مدیریت کارخانه ی خود به مشکل خورده است و نیازمند کمک شماست. این سوال شامل مدل سازی تعاملات اقتصادی بین دو گروه اصلی است: **افراد** و **محل های کار**.

می توانید کلاس های اولیه پروژه را از [این لینک](#) دانلود کنید.

- **افراد:** در این مدل، افراد با خصوصیت هایی مانند نام، سن و سطح تجربه (*Level*) تعریف می شوند. افراد دارای حرفه های مختلفی هستند (مانند مهندس، کارگر و معلم) که هر حرفه ممکن است ویژگی ها و منطق درآمد و هزینه خاص خود را داشته باشد. هر فرد هزینه های زندگی روزانه دارد و در صورت اشتغال، می تواند درآمدی نیز تولید کند.
- **محل های کار:** محل های کار نیز موجودیت هایی هستند که با نام، قدمت و سطح (*Level*) مشخص می شوند. انواع مختلفی از محل های کار وجود دارند (مانند معدن، مدرسه و شرکت) که هر نوع می تواند ظرفیت، هزینه نگهداری و تأثیر متفاوتی بر درآمد افراد شاغل در آن داشته باشد. محل های کار می توانند افراد با حرفه های مرتبط را استخدام کرده و آن ها را در خود جای دهند.

در این سوال تمرکز اصلی در این بخش، طراحی و پیاده سازی کلاس ها برای مدل سازی این موجودیت ها و رفتارهای آن ها است. شما کلاس های پایه *Person* و *WorkPlace* و سپس کلاس های مشتق شده ی *Teacher*، *Engineer* و *Worker* (از *Person*) و *School*، *Mine* و *Company* (از *WorkPlace*) را پیاده سازی خواهید کرد. این کلاس ها باید شامل ویژگی ها و متدهایی باشند که منطق مربوط به هر موجودیت را در بر می گیرند، از جمله:

- محاسبه ی درآمد روزانه افراد (بر اساس حرفه، سطح فرد، و سطح محل کار).
- محاسبه ی هزینه های زندگی افراد و هزینه های نگهداری محل های کار.
- منطق ارتقاء سطح برای افراد و محل های کار.
- منطق استخدام افراد در محل های کار با در نظر گرفتن ظرفیت.
- محاسبه کلی درآمد و هزینه ی تولید شده توسط تمام افراد یا تمام محل های کار.

جزئیات پروژه

▼ ثابت‌های پروژه: کلاس Consts

در طول پروژه، از مقادیر ثابتی مانند قیمت‌های پایه برای محاسبه‌ی هزینه‌ها استفاده می‌کنیم. برای سازماندهی این مقادیر، یک کلاس ساده به نام Consts تعریف می‌شود که این مقادیر ثابت را در خود نگه می‌دارد.

```

1 public class Consts {
2     public static final int MINEBasePrice = 150;
3     public static final int SCHOOLBasePrice = 100;
4     public static final int COMPANYBasePrice = 90;
5
6     public static final int BasePlaceCost = 2500;
7     public static final int LevelMul = 50;
8
9     public static final int WORKERBasePrice = 1200;
10    public static final int TEACHERBasePrice = 1500;
11    public static final int ENGINEERBasePrice = 2000;
12
13    public static final int WORKERBaseCost = 200;
14    public static final int TEACHERBaseCost = 150;
15    public static final int ENGINEERBaseCost = 300;
16
17    public static final int WORKERBaseIncome = 800;
18    public static final int TEACHERBaseIncome = 300;
19    public static final int ENGINEERBaseIncome = 1000;
20
21    public static final int MinAge = 15;
22    public static final int AgeMul = 10;
23 }
```

کلاس Person

ویژگی‌ها

- **instances**: یک متغیر static است، به این معنی که تنها یک کپی از آن وجود دارد که بین تمام اشیاء (نمونه‌های) ساخته شده از کلاس Person به اشتراک گذاشته می‌شود. هدف آن نگهداری لیستی از تمام اشیاء Person است که تاکنون ایجاد شده‌اند.

- **name** : نام فرد و از نوع رشته است.
- **age** : سن فرد و از نوع عدد صحیح است.
- **level** : سطح تجربه یا مهارت فرد است. از نوع عدد صحیح است و با مقدار اولیه‌ی ۱ شروع می‌شود.
- **job** : نوع شغل فرد است، از نوع رشته است و با مقدار اولیه‌ی "" (رشته‌ی خالی) شروع می‌شود.
- **workPlace** : شیء محل کار که فرد در آن مشغول است. از نوع Workplace است؛ اما با مقدار اولیه‌ی None شروع می‌شود.

دقت داشته باشید که در دنیای علی فقط فقط شغل های Teacher Worker و Engineer وجود دارد و قابلیت ساخت Person از نوع دیگری نداریم.

متدها

متد سازنده:

- پارامترهای name و age برای مقداردهی اولیه ویژگی‌های متناظر شیء استفاده می‌شوند.
- ویژگی level با مقدار پیش‌فرض 1 مقداردهی می‌شود.
- ویژگی‌های job و work_place به ترتیب با "" و Null مقداردهی اولیه می‌شوند، زیرا فرد در لحظه‌ی ایجاد هنوز شغل یا محل کار مشخصی ندارد.
- شیء فعلی باید به لیست instances اضافه شود تا ردیابی تمام افراد امکان‌پذیر باشد.

متد `public double doLevel(int income)`

این متد مسئول اعمال تأثیر سطح فرد و سطح محل کار بر روی درآمد پایه‌ی فرد (income) است. این متد مقدار نهایی درآمد تعدیل‌شده با سطح را برمی‌گرداند. فرمول محاسبه به شرح زیر است:

$$income \times \sqrt{level \times workplace.level}$$

نکته: برای پیاده‌سازی این متد (و متد calc که از آن استفاده می‌کند) در این تمرین، فرض بر این است که این متد تنها برای افراد شاغل فراخوانی می‌شود یا منطق کل برنامه این وضعیت را قبل از فراخوانی مدیریت می‌کند.

متدهای `public TODO int calcIncome()` و `public TODO int`

`calcLifeCost()`

این دو متد برای محاسبه‌ی درآمد روزانه و هزینه‌ی زندگی فرد در نظر گرفته شده‌اند. در کلاس پایه‌ی `Person` ، پیاده‌سازی این متدها انجام نمی‌شود؛ زیرا منطق واقعی محاسبه درآمد و هزینه زندگی به نوع شغل فرد وابسته است (مثلاً یک مهندس با یک معلم متفاوت درآمد دارد). پیاده‌سازی اصلی این متدها در زیرکلاس‌های `Engineer` ، `Teacher` و `Worker` انجام خواهد شد. دقت داشته باشید برای `Person` که معلم یا مهندس یا کارگر نباشد انجام این متد ممکن نیست.

متد `public double calc()`

این متد وضعیت مالی خالص روزانه فرد را محاسبه می‌کند. برای این کار، ابتدا درآمد پایه‌ی فرد را و هزینه‌ی زندگی او را به دست می‌آورد. سپس درآمد پایه را بر اساس سطح فرد و محل کار تعدیل می‌کند. در نهایت، هزینه‌ی زندگی را از درآمد تعدیل‌شده کم کرده و نتیجه (درآمد خالص یا زیان) را برمی‌گرداند.

متد `public static double calcAll()`

هدف این متد، محاسبه‌ی مجموع وضعیت مالی روزانه تمام افرادی است که در مدل ما وجود دارند.

متد `public String getJob()`

این متد، رشته‌ی شغل فعلی فرد را برمی‌گرداند.

متد `public void upgrade()`

این متد سطح تجربه‌ی فرد (`level`) را یک واحد افزایش می‌دهد. این متد هیچ مقداری را برنمی‌گرداند.

پس از پیاده‌سازی کلاس بنیادین `Person` ، اکنون نوبت به کلاس دیگری از موجودیت‌های اصلی مدل ما، یعنی کلاس `WorkPlace` (محل کار)، می‌رسد. این کلاس نقش مدل‌سازی مکان‌هایی را دارد که افراد در آن‌ها مشغول به کار می‌شوند. کلاس `WorkPlace` ویژگی‌ها و رفتارهای مشترک انواع مختلف محل کار (مانند معدن، مدرسه و شرکت) را تعریف می‌کند و مانند کلاس `Person` ، پایه‌ای برای زیرکلاس‌های خود خواهد بود.

کلاس `WorkPlace`

ویژگی‌ها

- **instances** : یک متغیر کلاس است که لیستی از تمام اشیاء Workplace را که تاکنون ایجاد شده‌اند، نگهداری می‌کند. هر شیء جدید Workplace هنگام ساخت به این لیست اضافه می‌شود.
 - **name** : نام محل کار است و از نوع رشته است.
 - **level** : سطح فعلی محل کار است. از نوع عدد صحیح است و با مقدار اولیه‌ی ۱ شروع می‌شود.
 - **expertise** : نوع تخصص محل کار (مانند "mine" ، "school" ، "company") است. از نوع رشته است و با مقدار اولیه‌ی "" (رشته خالی) شروع می‌شود و در زیرکلاس‌ها مقداردهی می‌شود.
 - **employees** : آرایه‌ای از اشیاء Person که در این محل کار مشغول هستند. با لیست خالی [] شروع می‌شود.
 - **employeeCount** : تعداد کارمندانی که در محل کار مشغول هستند.
 - **capacity** : حداکثر تعداد افرادی که می‌توانند در این محل کار مشغول باشند. از نوع عدد صحیح است، با مقدار اولیه‌ی ۱ شروع می‌شود و با ارتقاء سطح به‌روزرسانی می‌شود.
- دقت داشته باشید در دنیای علی محل‌های کار صرفاً Mine ، School و Company هستند و امکان ساخت محل کار از نوع دیگری وجود ندارد.

متدها

متد سازنده:

- پارامتر name برای مقداردهی ویژگی name شیء استفاده می‌شود.

متد `public int getPrice()`

این متد هزینه‌ی ساخت این محل کار را بر اساس نوع تخصص آن (expertise) از کلاس Consts با استفاده از BasePrice برمی‌گرداند.

متد `public void upgrade()`

این متد سطح محل کار را یک واحد افزایش می‌دهد. پس از افزایش سطح، متد calcCapacity را فراخوانی می‌کند تا ظرفیت محل کار بر اساس سطح جدید محاسبه و به‌روزرسانی شود.

متد `public void hire(Person person)`

این متد مسئول استخدام یک فرد (`person` که یک شیء `Person` است) در این محل کار است. قبل از استخدام، ظرفیت فعلی محل کار با تعداد کارمندان فعلی مقایسه می‌شود. اگر محل کار پر نباشد (تعداد کارمندان کمتر از ظرفیت)، شیء `person` به لیست `employees` اضافه می‌شود و ویژگی `workplace` شیء `person` برابر با این محل کار قرار داده می‌شود تا فرد به محل کار متصل شود؛ در غیر این صورت پیام: `this.name is full` چاپ می‌شود.

متد `public TODO void calcCapacity()`

این متد مسئول محاسبه‌ی ظرفیت حداکثر کارمند برای محل کار بر اساس سطح فعلی آن است. منطق محاسبه‌ی ظرفیت برای انواع مختلف محل کار متفاوت است (مثلاً معدن ممکن است با مدرسه فرق داشته باشد). بنابراین، پیاده‌سازی واقعی این متد در زیرکلاس‌های `Mine`، `School` و `Company` انجام خواهد شد. دقت داشته باشید انجام این متد برای محل‌هایی به جز `Mine`، `School` و `Company` ممکن نیست.

متد `public TODO void calcCosts()`

این متد مسئول محاسبه‌ی هزینه‌های نگهداری روزانه برای این محل کار است. مجدداً منطق محاسبه‌ی هزینه نیز برای انواع مختلف محل کار متفاوت است. مانند `calcCapacity`، پیاده‌سازی واقعی این متد در زیرکلاس‌ها انجام خواهد شد.

متد `public String getExpertise()`

این متد صرفاً مقدار ویژگی `expertise` را که نشان‌دهنده نوع تخصص محل کار است، برمی‌گرداند.

متد `public int calc()`

این متد وضعیت مالی خالص روزانه‌ی مربوط به خود محل کار را محاسبه می‌کند. معمولاً محل‌های کار درآمدی تولید نمی‌کنند (این افراد هستند که درآمد تولید می‌کنند)، بلکه صرفاً هزینه دارند. بنابراین، این متد منفی هزینه‌ی نگهداری روزانه را که توسط متد `calcCosts()` محاسبه می‌شود، برمی‌گرداند.

متد `public static int calcAll()`

این متد به عنوان یک متد ایستا (*Static Method*) تعریف شده است و مسئول محاسبه‌ی مجموع وضعیت مالی روزانه‌ی تمام محل‌های کاری است که در مدل ما وجود دارند.

در بخش بعدی از پروژه، شما باید سه کلاس به نام‌های Mine (معدن)، School (مدرسه)، و Company (شرکت) را تعریف کنید. هر سه کلاس باید از کلاس والد Workplace (محل کار) ارث ببرند. برای هر کلاس، متد سازنده و دو متد calcCapacity و calcCosts را طبق دستورالعمل‌های زیر پیاده‌سازی کنید.

کلاس‌های Mine ، School و Company

برای هریک از کلاس‌های Mine ، School و Company ، موارد زیر را پیاده‌سازی کنید.

متد سازنده:

متد سازنده در هر سه کلاس باید به شکل زیر باشد:

- تنها یک آرگومان ورودی به نام name را بپذیرد.
- ابتدا با استفاده از super() ، متد سازنده کلاس والد (WorkPlace) را با ارسال آرگومان name فراخوانی کند.
- سپس، صفت expertise را بر اساس نوع کلاس مقداردهی کند:

◦ معدن (Mine): "mine"

◦ مدرسه (School): "school"

◦ شرکت (Company): "company"

متد public void calcCapacity()

این متد باید ظرفیت محل کار را بر اساس صفت level محاسبه کرده و نتیجه را در صفت capacity ذخیره کند. مقدار capacity باید از نوع int (عدد صحیح) باشد. این متد هیچ مقداری را برنمی‌گرداند. فرمول‌های محاسبه‌ی ظرفیت برای هر نوع محل کار به شرح زیر است (مقدار level یک عدد صحیح مثبت است).

- **معدن (Mine):

$$capacity = level^2$$

• ****مدرسه (School)**:

$$capacity = \lfloor \sqrt{level} \rfloor$$

• ****شرکت (Company)**:

$$capacity = level$$

متد `public int calcCosts()`

این متد باید هزینه‌های محل کار را بر اساس صفت `level` و ثابت‌های موجود در کلاس `Consts` محاسبه کرده و مقدار محاسبه‌شده را برگرداند. مقدار بازگشتی این متد باید از نوع `int` باشد.

فرمول‌های محاسبه‌ی هزینه‌ها برای هر نوع محل کار به شرح زیر است (مقدار `level` یک عدد صحیح مثبت است).

• ****معدن (Mine)**:

$$costs = Consts.BasePlaceCost + Consts.LevelMul \times level$$

• ****مدرسه (School)**:

$$costs = Consts.BasePlaceCost \times \lfloor level \rfloor$$

• ****شرکت (Company)**:

$$costs = Consts.BasePlaceCost \times level$$

کلاس‌های Teacher ، Engineer و Worker

در بخش بعدی از پروژه از شما خواسته می‌شود سه کلاس به نام‌های `Worker` (کارگر)، `Teacher` (معلم) و `Engineer` (مهندس) را تکمیل کنید. این سه کلاس باید از کلاس `Person` (شخص) ارث ببرند.

برای هر یک از این کلاس‌های فرزند، لازم است موارد زیر را پیاده‌سازی کنید:

متدها

۱. سازنده

۲. متد `getPrice` : محاسبه‌ی هزینه‌ی استخدام

۳. متد `calcLifeCost` : محاسبه‌ی هزینه‌های زندگی

۴. متد `calcIncome` : محاسبه‌ی درآمد

در ادامه، جزئیات پیاده‌سازی هر بخش شرح داده شده‌است.

برای هر یک از کلاس‌های `Worker` ، `Teacher` و `Engineer` ، موارد زیر را پیاده‌سازی کنید.

متد سازنده

متد سازنده در هر سه کلاس باید به شکل زیر باشد:

- دو آرگومان `name` و `age` دریافت می‌شوند.
- ابتدا، سازنده‌ی کلاس والد (`Person`) را برای آرگومان‌های `name` و `age` فراخوانی می‌کند.
- سپس، ویژگی `job` در هر کلاس مطابق با تخصص آن کلاس تنظیم شود:

◦ کلاس (`Worker`) : `"worker"`

◦ کلاس (`Teacher`) : `"teacher"`

◦ کلاس (`Engineer`) : `"engineer"`

متد `public int getPrice()`

این متد باید هزینه‌ی استخدام شخص را بر اساس تخصص (`job`) و سن او (`age`) محاسبه کرده و بازگرداند. مقدار بازگشتی این متد باید از نوع `int` باشد.

فرمول محاسبه‌ی هزینه برای هر تخصص به شرح زیر است.

- ****کارگر (`Worker`) :**

$$price = \left\lfloor \frac{Consts.WORKERBasePrice \times Consts.MinAge}{age} \right\rfloor$$

• **معلم (Teacher):

$$costs.TEACHERBasePrice - (age - Consts.MinAge) \times Consts.AgeMul$$

• **مهندس (Engineer):

$$price = \lfloor Consts.ENGINEERBasePrice \times \sqrt{\frac{Consts.MinAge}{age}} \rfloor$$

متد `public double calcLifeCost()`

این متد باید مقدار مخارج زندگی شخص را بر اساس تخصص (job) و سن او (age) محاسبه کرده و بازگرداند. مقدار بازگشتی این متد باید از نوع `int` باشد.

فرمول محاسبه برای هر تخصص به شرح زیر است.

• **کارگر (Worker):

$$costs = \lfloor \frac{Consts.WORKERBaseCost \times age}{Consts.MinAge} \rfloor$$

• **معلم (Teacher):

$$costs.TEACHERBaseCost + (age - Consts.MinAge) \times Consts.AgeMul$$

• **مهندس (Engineer):

$$costs = \lfloor Consts.ENGINEERBaseCost \times \sqrt{\frac{age}{Consts.MinAge}} \rfloor$$

متد `public double calcIncome()`

این متد باید درآمد شخص را بر اساس تخصص (job)، سن (age) و محل کار او (workplace) محاسبه کرده و بازگرداند. مقدار بازگشتی این متد باید از نوع `int` باشد.

فرمول محاسبه برای هر تخصص به شرح زیر است.

• **کارگر (Worker):

$$Income = \lfloor \frac{Consts.WORKERBaseIncome \times Consts.MinAge}{age} \rfloor$$

• **معلم (Teacher):

$$ts.TEACHERBaseIncome - (age - Consts.MinAge) \times Consts.AgeMul$$

• **مهندس (Engineer):

$$Income = \lfloor Consts.ENGINEERBaseIncome \times \sqrt{\frac{Consts.MinAge}{age}} \rfloor$$

مثال

ورودی نمونه ۱

```
public class Main {
    public static void main(String[] args) {

        // Create workplaces
        Mine mine = new Mine("Gold Mine");
        School school = new School("MIT");
        Company company = new Company("Apple");

        // Create people
        Worker worker = new Worker("John", 25);
        Teacher teacher = new Teacher("Emma", 35);
        Engineer engineer = new Engineer("Alex", 28);

        // Hire people
        mine.hire(worker);
        school.hire(teacher);
        company.hire(engineer);

        System.out.println("=== EMPLOYMENT STATUS ===");
        System.out.println(worker.getName() + " works at " + worker.getWorkPlace());
        System.out.println(teacher.getName() + " works at " + teacher.getWorkPlace());
        System.out.println(engineer.getName() + " works at " + engineer.getWorkPlace());
    }
}
```

```

        System.out.println("\n=== DAILY NET BALANCE ===");
        System.out.println(worker.getName() + ": $" + worker.calc());
        System.out.println(teacher.getName() + ": $" + teacher.calc());
        System.out.println(engineer.getName() + ": $" + engineer.calc());

        // Upgrade worker and workplace
        worker.upgrade();
        mine.upgrade();

        System.out.println("\n=== AFTER UPGRADES ===");
        System.out.println(worker.getName() + " (level " + worker.getLevel() + "):
        System.out.println(mine.getName() + " (level " + mine.getLevel() + ", capac

        System.out.println("\n=== TOTAL SYSTEM BALANCE ===");
        System.out.println("Total: $" + Person.calcAll());

        System.out.println("\n=== DETAILED BREAKDOWN ===");
        System.out.println("Worker - Income: $" + worker.calcIncome() + ", Cost: $"
        System.out.println("Teacher - Income: $" + teacher.calcIncome() + ", Cost:
        System.out.println("Engineer - Income: $" + engineer.calcIncome() + ", Cost

    }
}

```

خروجی نمونه ۱

```

=== EMPLOYMENT STATUS ===
John works at Gold Mine
Emma works at MIT
Alex works at Apple

=== DAILY NET BALANCE ===
John: $147.0
Emma: $-250.0
Alex: $322.0

=== AFTER UPGRADES ===
John (level 2): $627.0
Gold Mine (level 2, capacity 4)

=== TOTAL SYSTEM BALANCE ===
Total: $699.0

```

=== DETAILED BREAKDOWN ===

Worker - Income: \$480, Cost: \$333

Teacher - Income: \$100, Cost: \$350

Engineer - Income: \$731, Cost: \$409

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که شامل کلاس های زیر باشد:

Person.java

WorkPlace.java

Worker.java

Teacher.java

Engineer.java

Mine.java

Company.java

School.java